Tema 4: Interfaces Gráficas con Swing
Parte I - Lo Básico

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Qué es una GUI

- Una interfaz gráfica de usuario (GUI) es la interfaz entre el usuario y la aplicación.

Qué es una GUI

- Una interfaz gráfica de usuario (GUI) es la interfaz entre el usuario y la aplicación.
- Presenta las acciones disponibles al usuario y le permite interactuar con el programa mediante la manipulación de los componentes (por ejemplo, hacer un click sobre un botón).

Qué es una GUI

- Una interfaz gráfica de usuario (GUI) es la interfaz entre el usuario y la aplicación.
- Presenta las acciones disponibles al usuario y le permite interactuar con el programa mediante la manipulación de los componentes (por ejemplo, hacer un click sobre un botón).
- Dos maneras de programar GUIs:
  - **La Fácil**: usar una GUI para generar GUIs
  - **La Difícil**: escribir todo el código para crear componentes y conectaros con "listeners", etc.

Qué es una GUI

- Una interfaz gráfica de usuario (GUI) es la interfaz entre el usuario y la aplicación.
- Presenta las acciones disponibles al usuario y le permite interactuar con el programa mediante la manipulación de los componentes (por ejemplo, hacer un click sobre un botón).
- Dos maneras de programar GUIs:
  - **La Fácil**: usar una GUI para generar GUIs
  - **La Difícil**: escribir todo el código para crear componentes y conectaros con "listeners", etc.
- Vamos a seguir la manera **Difícil**, así se aprende más!

GUI en Java

- The Abstract Window Toolkit (**AWT**)
  1. Java Core version 1.0, en 1990
  2. Usa "widgets" nativos para mostrar las componentes (dependientes del SO)
  3. Se ve diferente en distintas plataformas
  4. Los widgets tienen que existir en todas las plataformas

GUI en Java

- The Abstract Window Toolkit (**AWT**)
  1. Java Core version 1.0, en 1990
  2. Usa "widgets" nativos para mostrar las componentes (dependientes del SO)
  3. Se ve diferente en distintas plataformas
  4. Los widgets tienen que existir en todas las plataformas
- **Swing** (Java Foundation Classes – JFC)
  1. Proyecto iniciado por Sun y Netscape en 1997
  2. Implementa los widgets de forma independiente al SO
  3. Se ve igual en todas las plataformas
- SWT, SwingX, JGoodies, JavaFX, Apache Pivot, Qt Jambi,...

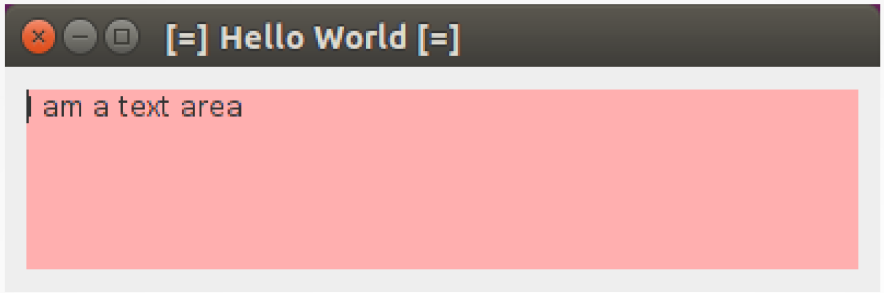
Empezando con Swing

- **Para programar una aplicación usando Swing, debemos realizar los siguientes pasos**:
  - Elegir un "contenedor" de primer nivel para colocar los componentes.
  - Fijar un layout, que determine la disposición en las que aparecerán las componentes. El layout divide el contenedor en distintas partes, y en cada una de ellas podemos poner una componente.
    Puede ser **null, BorderLayout, GridLayout, etc.**
  - Colocar las componentes.
    Si queremos colocar más de una componente en una de las zonas establecidas por el layout, entonces usaremos paneles. Un **panel** es una componente y un contenedor simultáneamente.
- Primero debemos elegir un tipo de "contenedor". En Swing tenemos tres tipos de ventana:
  1. Frame
  2. Dialog
  3. Applet (para aplicaciones Web)
- El objeto de Swing que representa un "frame" se llama **JFrame**.
- En Swing, los nombres de las componentes tienen una **J** al inicio (JButton, JLabel, etc.).

Empezando con Swing

- Cada interfaz gráfica comienza con una ventana principal, en la que se muestran sus componentes.
- Primero debemos elegir un tipo de "contenedor". En Swing tenemos tres tipos de ventana:
  1. Frame
  2. Dialog
  3. Applet (para aplicaciones Web)
- El objeto de Swing que representa un "frame" se llama **JFrame**.
- En Swing, los nombres de las componentes tienen una **J** al inicio (JButton, JLabel, etc.).

Ejemplos de componentes



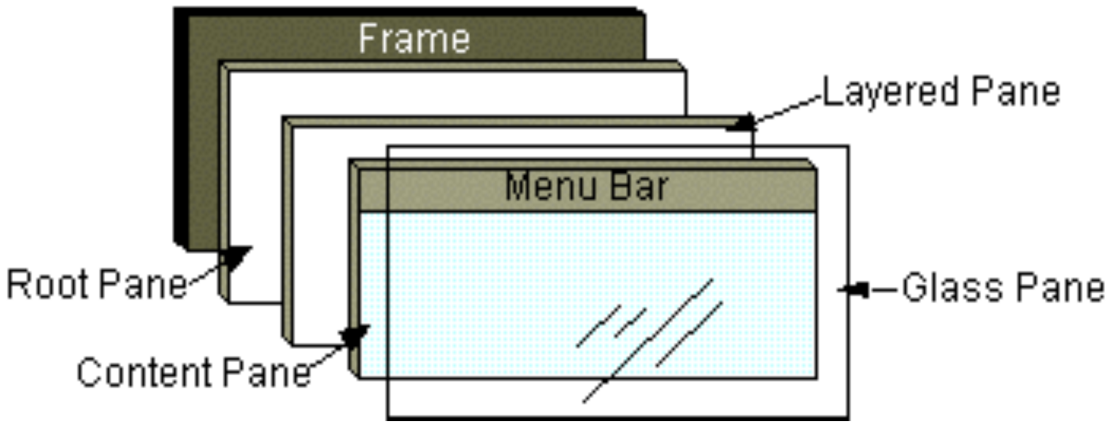
JFrame con "layout" **null** y una componente **JTextArea**



JDialog con "layout" **null**, una componente **JLabel** y dos componentes **JButton**

Frame en Swing: JFrame

- La clase para crear un "frame" en Swing se llama **JFrame**. Ya hemos visto un ejemplo. Representa una ventana como:



- El **Content Pane** es donde colocamos los componentes, las otras partes (Panes) no son importantes de momento.

Nuestro primer JFrame!

- Para utilizar componentes de **Swing** tenemos que importar la librería *javax.swing.**
- Para crear un **JFrame** tenemos que crar un objeti de dicho tipo y hacerlo visible
- Para crear un **JFrame** podemos usar:

```
JFrame frame = new JFrame("[=] Hello World [=]");
```

- Para mostrar el **JFrame** hacemos:

```
frame.setVisible (true);
```

  Recuerda que por defecto no es visible
- El tamaño del **JFrame** se fija (en pixeles) a través del comando:

```
frame.setSize (ancho,alto );
```

- Por defecto la ventana aparece en la posición (0,0) siendo la esquina superior izquierda. Ten en cuenta que crece hacia la derecha y hacia abajo. Para fijar la localización de la ventana podemos usar:

```
frame.setLocation (x, y);  frame.setBounds(x,y, ancho,  alto );
```

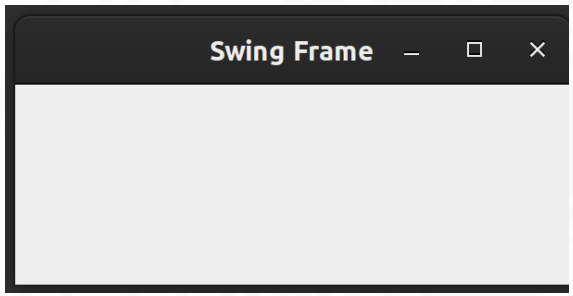
Nuestro primer JFrame!

```
import javax.swing.*;

public class FrameExample {

  public static void main(String[] args) {
    //Create a frame
    JFrame frame = new JFrame("Swing Frame");
    //Locates the frame at position (400,400) on the
    //screen, with width 600 and height 300
    frame.setBounds(400,400,600,300);
    //Display the frame
    frame.setVisible (true);
  }

}
```



Nuestro primer JFrame!

- Una ventana se puede cerrar de distintas formas, utilizando el método:

```
frame.setDefaultCloseOperation(JFrame_constant);
```

  donde JFrame_constant puede tomar uno de los siguientes valores:
  - **JFrame.DO_NOTHING_ON_CLOSE**: No hace nada cuando el usuario cierra la ventana.
  - **JFrame.HIDE_ON_CLOSE**: Valor por defecto. Esconde la ventana cuando el usuario la cierra. La ventana pasa a ser invisible pero el programa se sigue ejecutando.
  - **JFrame.DISPOSE_ON_CLOSE**: Cierra la ventana y libera todos los recursos ocupados por ella.
  - **JFrame.EXIT_ON_CLOSE**: Cierra la aplicación.

Nuestro primer JFrame!

- Es más común crear clases que hereden de **JFrame**. Es decir, podemos crearnos nuestras propias ventanas y utilizarlas en distintas aplicaciones.
- En general esto se puede hacer con cualquier componente.

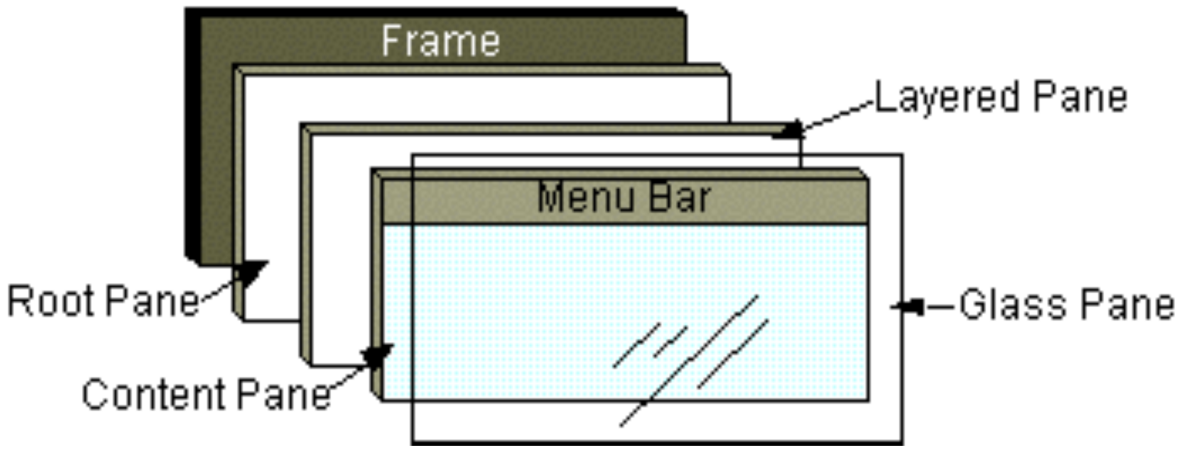
```
import javax.swing.*;

public class FrameExample extends JFrame{
  public FrameExample(){
    super("Swing Frame");
    initGUI();
  }
  private void initGUI() {
    this.setBounds(400,400,600,300);
    this.setVisible(true);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        new FrameExample();}
    });
  }
}
```

- invokeLater crea una tarea para ejecutar la ventana en la hebra de Swing

Colocando componentes en un JFrame

- Un **Frame** contiene en su parte superior, una zona exclusiva para colocar una barra de menú
- En un **Frame** se pueden colocar componentes (etiquetas, botones, campos de texto, etc.) utilizando su componente **content pane** (panel de contenido)



Colocando componentes en un JFrame

- Para acceder al content pane de un frame se puede usar:

```
Container contentPane = frame.getContentPane();
```

- Para añadir componentes se utiliza:

```
contentPane.add(componente);
```

- En el caso de los frames se puede utilizar:

```
frame.add(componente);
```

```
...
  private void initGUI() {
    this.setBounds(400,400,600,300);
    JLabel label = new JLabel();
    label.setSize(600,300);
    label.setLocation(0,0);
    label.setBackground(Color.YELLOW);
    label.setOpaque(true);
    this.add(label);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setVisible(true);
  }
...
```

Colocando componentes en un JFrame

- Java ofrece la posibilidad de fijar un **layout**, que determina la disposición de las componentes en la ventana
- De momento utilizaremos **null** (frame.setLayout(null)), que permite colocar las componentes de forma manual. Esto obliga a:
  1. Dar tamaño a las componentes que vamos a colocar (componente.setSize(x,y))
  2. Fijar su localización de forma manual indicando las coordenadas de la ventana donde queremos que aparezca (componente.setLocation(x,y))
  3. **Cuando se redimensiona la ventana las componentes no se mueven, quedan ancladas**. Esto puede hacer que dejen de verse.
  4. El cambio de sistema operativo, resolución de la pantalla, fuente de letra, etc. puede afectar a como se visualicen los componentes

Usa JPanel para Organizar Widgets

- Imagina la GUI como un árbol: **Si A está colocado sobre B significa que A es hijo de B**
- Se puede colocar los widgets de manera absoluta, respecto a la esquina superior izquierda (0,0) del padre
- Poder agrupar los widgets es fundamental, nos permite...
  1. manejar conjunto de componentes
  2. mover varios a la vez
  3. definir propiedades comunes
- **JPanel** nos permite agrupar componentes

Usa JPanel para Organizar Widgets

Este código crea una instancia de JPanel, lo hace opaco (no transparente. Esto no hace falta siempre) y lo coloca en el Content Pane del JFrame

```
public class PanelExample extends JFrame {
  public PanelExample() {
    super("[=] There's a JPanel in here! [=]");
    initGUI();
  }

  private void initGUI() {
    JPanel mainPanel = new JPanel();
    mainPanel.setOpaque(true);
    this.setContentPane(mainPanel);

    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setSize(290, 100);
    this.setVisible(true);
  }
  ...
}
```

JPanels Anidados

```
private void initGUI() {
  JPanel mainPanel = new JPanel();
  mainPanel.setLayout(null);

  JPanel redPanel = new JPanel();
  redPanel.setBackground(Color.red);
  redPanel.setLocation(60, 10);
  redPanel.setSize(50, 50);
  mainPanel.add(redPanel);

  JPanel bluePanel = new JPanel();
  bluePanel.setBackground(Color.blue);
  bluePanel.setLocation(200, 10);
  bluePanel.setSize(50, 50);
  mainPanel.add(bluePanel);

  this.setContentPane(mainPanel);
  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  this.setSize(290, 100);
  this.setVisible(true);
}
```

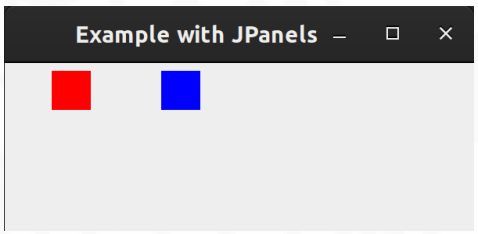
JPanels Anidados

```
private void initGUI() {
  JPanel mainPanel = new JPanel();
  mainPanel.setLayout(null);

  JPanel redPanel = new JPanel();
  redPanel.setBackground(Color.red);
  redPanel.setLocation(60, 10);
  redPanel.setSize(50, 50);
  mainPanel.add(redPanel);

  JPanel bluePanel = new JPanel();
  bluePanel.setBackground(Color.blue);
  bluePanel.setLocation(200, 10);
  bluePanel.setSize(50, 50);
  mainPanel.add(bluePanel);

  this.setContentPane(mainPanel);
  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  this.setSize(290, 100);
  this.setVisible(true);
}
```



Componente JLabel

- Un **JLabel** es una etiqueta que se suele utiloizar para mostrar texto informativo.
- **No se pueden editar directamente por el usuario**, solo se pueden modificar a través de código
- Existen distintas constructoras para crear etiquetas. Entre ellas, las más utilizadas son:
  1. **JLabel()**: Constructora por defecto. Texto vacío
  2. **JLabel(String text)**: Constructora que la crea utilizando el texto indicado
- Algunos métodos para las etiquetas son:
  1. **void setText(String)**: Pone texto a la etiqueta
  2. **String getText()**: Devuelve el texto de la etiqueta
  3. **void setFont(Font)**: Fija la fuente del texto de la etiqueta. Para crear la fuente basta usar **new Font(String, Font.type, int)** donde String es el tipo de fuente, type puede ser **BOLD, ITALIC** etc. e int el tamaño
  4. **void setHorizontalAlighment(JLabel.contant)**: Coloca el texto según lo especificado en la constante, que puede ser **CENTER, LEFT,** etc.

Componente JLabel

- Para consultar los tipos de fuentes disponibles, en forma de **String**, se puede utilizar:

```
String [] fontNames=GraphicsEnvironment.getLocalGraphicsEnvironment().
                                                    getAvailableFontFamilyNames();
System.out.println (Arrays. toString (fontNames));
```

- También se pueden utilizar las fuentes que aparecen como constantes estáticas del tipo **Font**. Por ejemplo Font.SANS_SERIF
- Para los estilos de fuente basta consultar los atributos estáticos de **Font**, de tipo entero. Por ejemplo Font.ITALIC

Componente JLabel

```
private static JLabel createLabel( int x, int y, int w, int h, String title , Font font,
                                   Color color ){

    JLabel label = new JLabel(title );
    label .setLocation (x, y);
    label .setSize (w,h);
    label .setHorizontalAlignment (JLabel. CENTER);
    label .setFont(font );
    label .setForeground(color );
    label .setOpaque(true);
    return label ;
  }
```

Componente JLabel

```
void initGUI(){
  super("Swing frame");
  this.setLayout(null);
  this.setBounds(400,400,250,100);
  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  Font redFont = new Font("Courier",Font.BOLD,15);
  JLabel redLabel = Example1.createLabel(10,0,50,40,"Red",redFont,Color.RED);
  Font yellowFont = new Font("Cabin",Font.ITALIC,15);
  JLabel yellowLabel = Example1.createLabel(70,0,50,40,"Yellow",yellowFont,Color.
      YELLOW);
  Font greenFont = new Font("Carlito",Font.BOLD,15);
  JLabel greenLabel = Example1.createLabel(130,0,50,40,"Green",greenFont,Color.GREEN);
  Font blueFont = new Font("SanSerif",Font.ITALIC,15);
  JLabel blueLabel = Example1.createLabel(190,0,50,40,"Blue",blueFont,Color.BLUE);

  this.add(redLabel);
  this.add(yellowLabel);
  this.add(greenLabel);
  this.add(blueLabel);
  this.setVisible (true);
}
```

Componente JLabel

```
void initGUI(){
  super("Swing frame");
  this.setLayout(null);
  this.setBounds(400,400,250,100);
  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  Font redFont = new Font("Courier",Font.BOLD,15);
  JLabel redLabel = Example1.createLabel(10,0,50,40,"Red",redFont,Color.RED);
  Font yellowFont = new Font("Cabin",Font.ITALIC,15);
  JLabel yellowLabel = Example1.createLabel(70,0,50,40,"Yellow",yellowFont,Color.
      YELLOW);
  Font greenFont = new Font("Carlito",Font.BOLD,15);
  JLabel greenLabel = Example1.createLabel(130,0,50,40,"Green",greenFont,Color.GREEN);
  Font blueFont = new Font("SanSerif",Font.ITALIC,15);
  JLabel blueLabel = Example1.createLabel(190,0,50,40,"Blue",blueFont,Color.BLUE);

  this.add(redLabel);
  this.add(yellowLabel);
  this.add(greenLabel);
  this.add(blueLabel);
  this.setVisible(true);
}
```



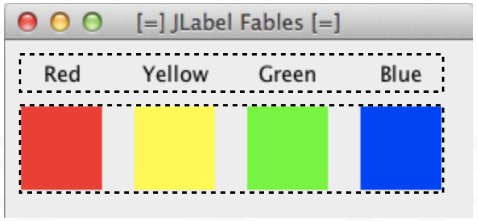
Componente JLabel

```
JPanel textPanel = new JPanel();
textPanel.setLayout(null);
textPanel.setLocation(10, 0);
textPanel.setSize(260, 30);
mainPanel.add(textPanel);

JLabel redLabel = new JLabel("Red");
redLabel.setLocation(0, 0);
redLabel.setSize(50, 40);
redLabel.setHorizontalAlignment(JLabel.CENTER);
textPanel.add(redLabel);
...

JPanel panelForPanels = new JPanel();
panelForPanels.setLayout(null);
panelForPanels.setLocation(10, 40);
panelForPanels.setSize(260, 50);
mainPanel.add(panelForPanels);

JPanel redPanel = new JPanel();
redPanel.setBackground(Color.red);
redPanel.setLocation(0, 0);
redPanel.setSize(50, 50);
panelForPanels.add(redPanel);
...
```



Componente JButton

- Representa un botón sobre el que se puede **hacer click y responder a un evento**
- Tiene varias constructoras, siendo las más utilizadas:

  JButton(): Crea un botón sin texto

  JButton(String text): Crea un botón con el texto especificado en text
- Como todas las componentes, un botón se puede habilitar o deshabilitar usando el método:

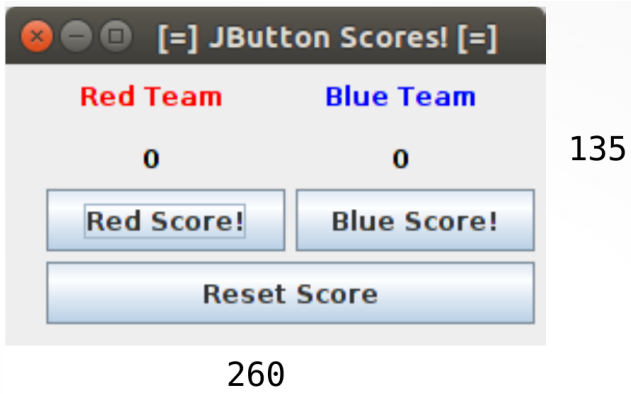
  void setEnabled(boolean)

  Por defecto todas las componentes están habilitadas. **Una componente habilitada responde a eventos.** Si está deshabilitada no responde a eventos aunque sea visible
- Toda componente se puede ocultar utilizando el método

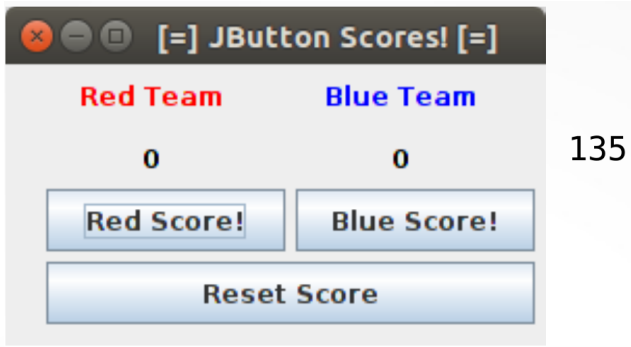
  void setVisible(boolean)

  En este caso se ocultan y no responden a eventos. **Por defecto las componentes son visibles**

Componente JButton

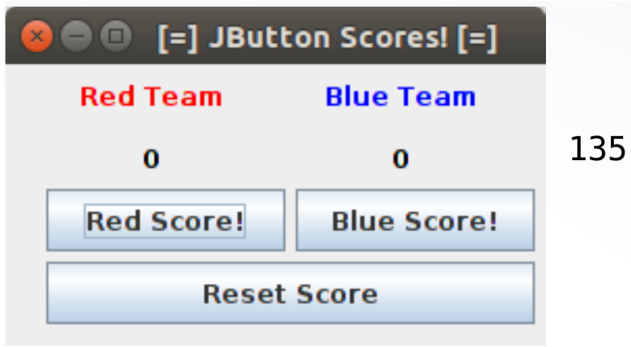


Componente JButton




```
public JLabel createLabel( int x, int y, int w, int h, String caption, Color color ){
    JLabel label = new JLabel(caption);
    label.setBounds(x,y,w,h);
    label.setHorizontalAlignment(JLabel.CENTER);
    label.setForeground(color);
    return label;
```

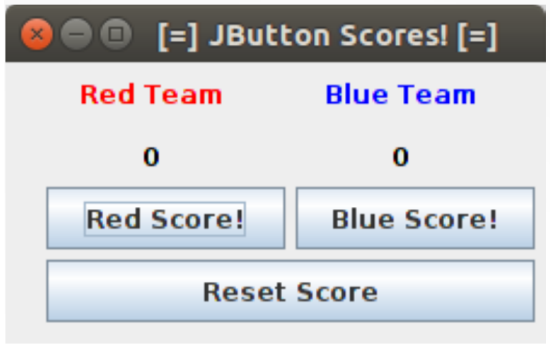
Componente JButton




```
public JLabel createLabel( int x, int y, int w, int h, String caption, Color color ){
    JLabel label = new JLabel(caption);
    label.setBounds(x,y,w,h);
    label.setHorizontalAlignment(JLabel.CENTER);
    label.setForeground(color );
    return label ;


public JButton createButton(String caption, int x, int y, int w, int h){
  JButton button = new JButton(caption);
  button.setBounds(x,y,w,h);
  return button;
}
```

Componente JButton



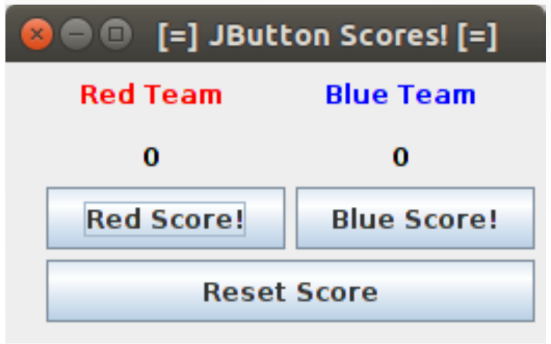

```
public void initGUI() {
  this.setLayout(null);
  JLabel redLabel = this.createLabel("Red Team",Color.red,10, 0,120, 30);
  this.add(redLabel);

  JLabel blueLabel = this.createLabel("Blue Team",Color.blue,130, 0,120, 30);
  this.add(blueLabel);

  this.redScore = this.createLabel("" + this.redScoreAmount, Color.black,10,30,120,30);
  this.add(redScore);

  this.blueScore = this.createLabel("" + this.blueScoreAmount,Color.black,130, 30,120, 30);
  this.add(blueScore);
  ...
```

Componente JButton




```
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.add(redButton);

    this.blueButton = this.createButton("Blue Score!",140, 60,115, 30);
    this.add(blueButton);

    this.resetButton = this.createButton("Reset Score",20, 95, 235, 30);
    this.add(resetButton);

    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setSize(260, 135);
    this.setVisible (true);
}
```

Componente JButton con Icono

- El botón se crea igual, y se le añade el icono con:

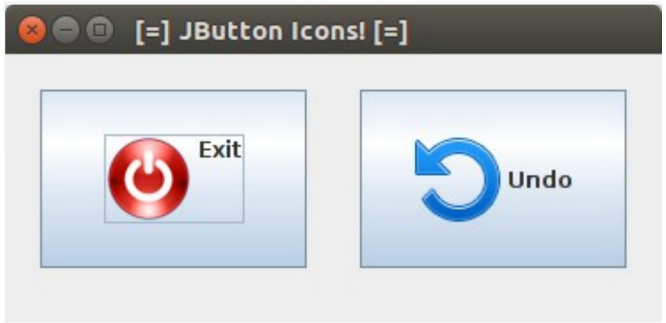
  boton.setIcon(new ImageIcon(path))

  Path hace referencia a la ruta donde se encuentra el icono. Si los iconos están en el mismo directorio que las carpetas **src** y **bin** entonces es suficiente con poner el nombre de la imagen
- El texto del botón se puede colocar en función de la imagen utilizando:

  boton.setVerticalTextPosition(JButton.CONSTANT-V)
  boton.setHorizontalTextPosition(JButoon.CONSTANT-H)

  CONSTANT-V puede ser **TOP, BOTTON,CENTER**
  CONSTANT-H puede ser **LEFT, CENTER, RIGHT**

Componente JButton con Icono




```
public void initGUI() {
  ...
  exitButton = new JButton("Exit");
  exitButton.setBounds(20,20,150,100);
  exitButton.setIcon(new ImageIcon("icons/exit.png"));
  exitButton.setVerticalTextPosition (JButton.TOP);
  exitButton.setHorizontalTextPosition (JButton.RIGHT);

  undoButton = new JButton("Undo");
  undoButton.setBounds(200, 20,150,100);
  undoButton.setIcon(new ImageIcon("icons/undo.png"));
  undoButton.setVerticalTextPosition (JButton.CENTER);
  undoButton.setHorizontalTextPosition (JButton.RIGHT);

  ...
}
```

Respondiendo a Eventos en Java

- En las aplicaciones GUI se utiliza un modelo de **programación dirigida por eventos**. Por ejemplo, cuando se pulsa un botón, cuando pasa el ratón por encima de una componente, cuando se cierra una ventana, etc, se genera un **evento**. La aplicación tendrá que determinar cómo responder a dicho evento.
- Para que una componente pueda responder a un evento, es necesario añadir un oyente (**listener**) a dicha componente.
- Existen distintos tipos de eventos. Comenzaremos por los eventos de tipo **Action**, correspondientes a la clase **ActionEvent**. Estos eventos responden a una acción del usuario.
- Por ejemplo, un botón debe registrar un **listener** a través del siguiente código:

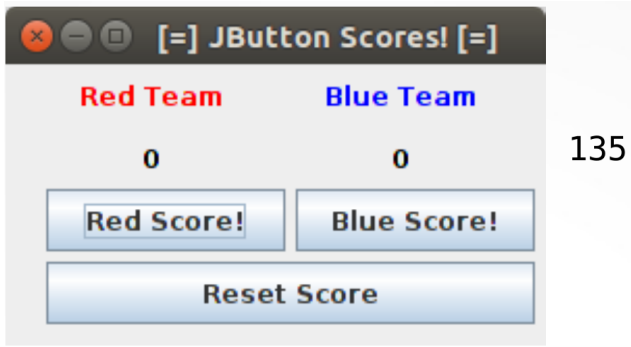
```
JButton button = new JButton(String text);
button.addActionListener( */objeto listener*/ )
```

Respondiendo a Eventos en Java

- El "objeto listener", que habitualmente es la ventana principal, debe responder al evento. Para ello tiene que implementar la interface **ActionListener**.

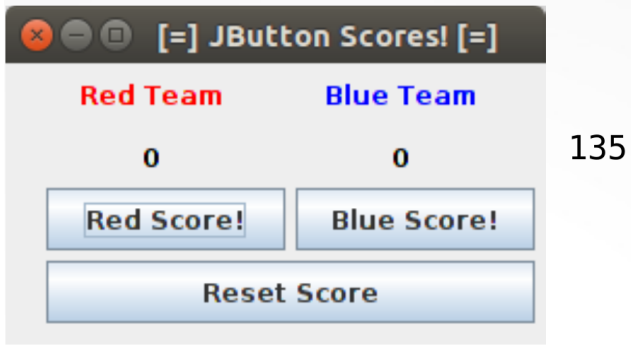
```
public interface ActionListener extends EventListener {
  /**
  * Invoked when an action occurs.
  */
  // Specifies what to do when the action occurs
  public void actionPerformed(ActionEvent e);
}
```

Respondiendo a Eventos en Java




```
public class Button2 extends JFrame implements ActionListener {
  ...
  public void initGUI() {
    ...
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.add(redButton);
    this.blueButton = this.createButton("Blue Score!",140, 60,115, 30);
    this.add(blueButton);
    this.resetButton = this.createButton("Reset Score",20, 95, 235, 30);
    this.add(resetButton);
    ...}
  public JButton createButton(String caption, int x, int y, int w, int h){
    ...
    button.addActionListener(this); //se añade el listener
    ...}
}
```

Respondiendo a Eventos en Java

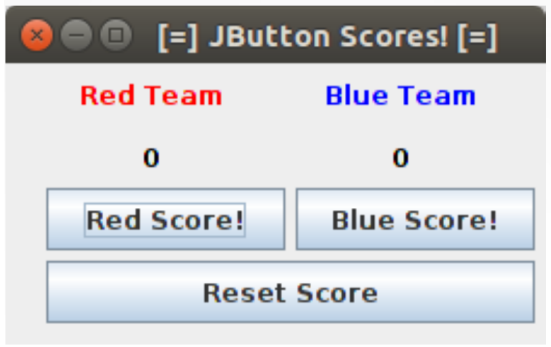



```
public class Button2 extends JFrame implements ActionListener {
  ...
  @Override
  public void actionPerformed(ActionEvent e) {
    if (e.getSource() == this.redButton) {
      this.redScoreAmount = this.redScoreAmount + 1;
      this.redScore.setText("" + this.redScoreAmount);
    } else if (e.getSource() == this.blueButton) {
      this.blueScoreAmount = this.blueScoreAmount + 1;
      this.blueScore.setText("" + this.blueScoreAmount);
    } else if (e.getSource() == this.resetButton) {
      this.redScoreAmount = 0;
      this.blueScoreAmount = 0;
      this.redScore.setText("" + this.redScoreAmount);
      this.blueScore.setText("" + this.blueScoreAmount);
    }
  }
}
```

Uso de clases anónimas/internas en los listeners

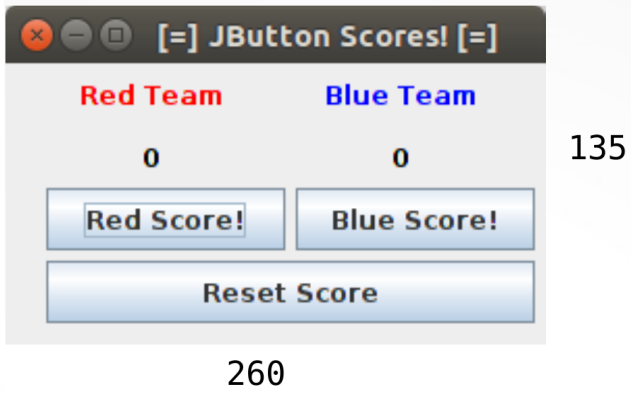
- Si una GUI tiene muchos componentes visuales que deben responder a eventos, el tamaño del método **actionPerformed** podría ser muy grande.
- Sería difícil de depurar ya que dicho código suele estar alejado del objeto que contiene el listener correspondiente. Si queremos cambiar la funcionalidad de un botón, tendríamos que buscar el lugar correspondiente de este método que hace referencia al botón, para modificarlo.
- Para tener un código más compacto y fácil de depurar es preferible usar clases anónimas y/o clases internas.
- Como norma podemos utilizar clases anónimas si el metodo **actionPerformed** es corto. En otro caso podríamos utilizar clases internas.

Uso de clases anónimas para los listeners




```
public class Button2 extends JFrame {
  public void initGUI() {
    ...
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.redButton.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        redScoreAmount = redScoreAmount + 1;
        redScore.setText("" + redScoreAmount);
      }
    });
    this.add(redButton);
  }
  ...
}
```

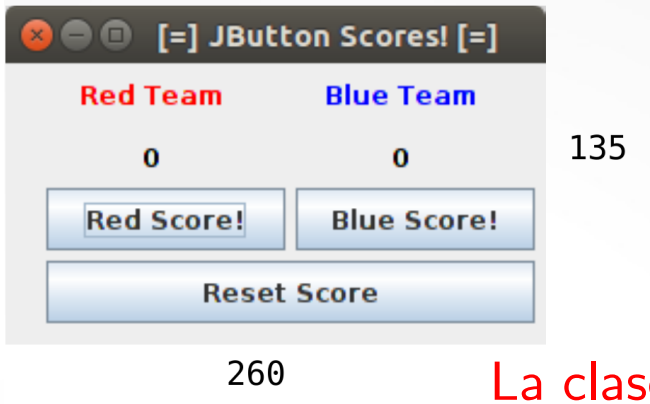
Uso de clases anónimas para los listeners




```
public class Button2 extends JFrame {
  public void initGUI() {
    ...
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.redButton.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        redScoreAmount = redScoreAmount + 1;
        redScore.setText("" + redScoreAmount);
      }
    });
    this.add(redButton);
  }
  ...
}
```

Cada botón tiene su "listener", no hace falta usar e.getSource()

Uso de clases anónimas para los listeners

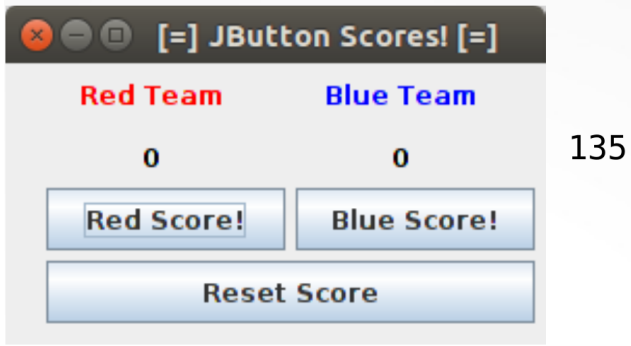


La clase no implementa la interfaz ActionListener, ya que ahora dicha clase no escucha ningún botón

```
public class Button2 extends JFrame {
  public void initGUI() {
    ...
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.redButton.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        redScoreAmount = redScoreAmount + 1;
        redScore.setText("" + redScoreAmount);
      }
    });
    this.add(redButton);
  }
  ...
}
```

Cada botón tiene su "listener", no hace falta usar e.getSource()

Uso de clases internas para los listeners




```
public class Button2 extends JFrame {

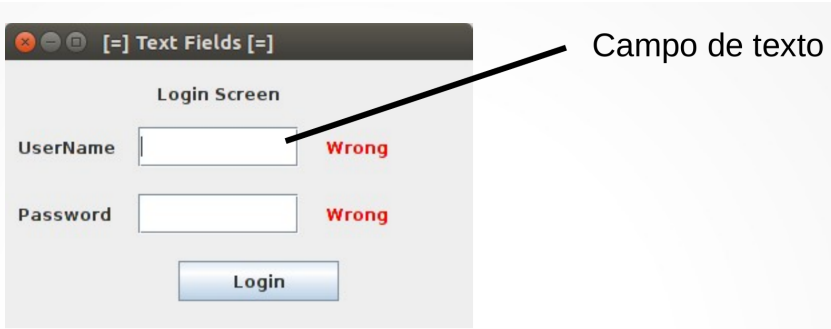
  class RedButtonListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent arg0) {
      redScoreAmount = redScoreAmount + 1;
      redScore.setText("" + redScoreAmount);
    }
  }

  public void initGUI() {
    ...
    this.redButton = this.createButton("Red Score!",20, 60,115, 30);
    this.redButton.addActionListener(new RedButtonListener());
    this.add(redButton);
  }
  ...
}
```

Componente JTextField

- Permite introducir texto al usuario
- Entre otras constructoras tiene:
  1. **JTextField()**: Crea un campo de testo vacío
  2. **JTextField(String text)**: Crea un campo de texto inicializado con la palabra text
  3. **JTextField(int i)**: Crea un campo de texto con capacidad para 20 caracteres
- Algunos métodos frecuentes:
  - String getText(): Devuelve el texto contenido en el campo de texto
  - void setEditable(boolean): Hace que el campo de texto sea editable o no
  - boolean isEditable(): Nos dice si el campo es editable o no

Componente JTextField




```
public void initGUI() {
  ...
  // user name text field
  userNameField = new JTextField(10);
  userNameField.setBounds(100,50,120,30);
  this.add(userNameField);
  ...
}
```

Componente JPasswordField

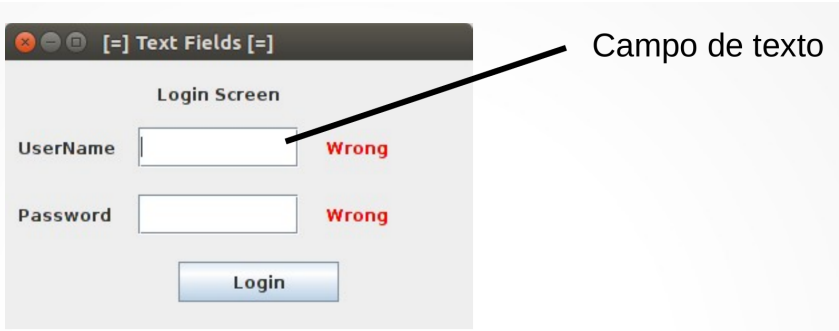
- Área de contraseña. Es un área de texto en la que los caracteres no se muestran
- La diferencia con un **JTextField** es que para conseguir la cadena de caracteres que ha puesto el usuario hay que usar la función:

passwordField.getPassword();

- Como es un campo editable, se puede indicar el caracter que queremos que salga cuando teclea un usuario usando el método:

passwordField.setEchoChar(char c)

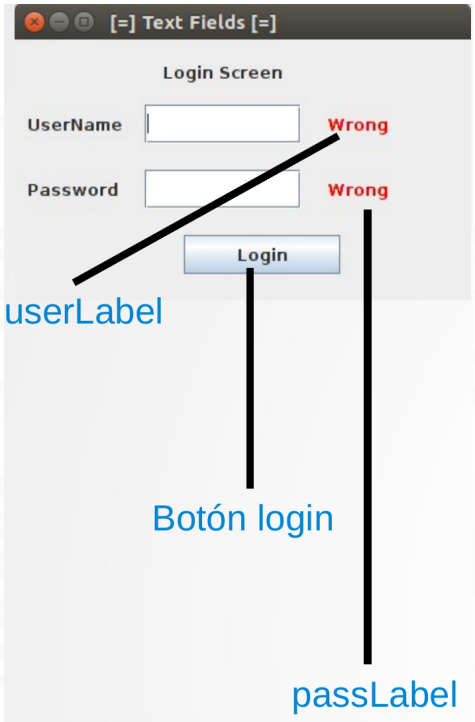
Componente JPasswordField




```
public void initGUI() {
  ...
  // password field
  loginField  = new JPasswordField(10);
  loginField .setBounds(100,100,120,30);
  loginField .setEchoChar('*');
  this.add(loginField );
  ...
}
```

Componente JPasswordField

```
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == login) {
    if (userNameField.getText().trim().compareTo("Bob") == 0) {
      userLabel.setForeground(Color.green);
      userLabel.setText("Correct!");
    } else {
      userLabel.setForeground(Color.red);
      userLabel.setText("Wrong!");
    }
    char[] answer = { 'R', 'o', 'b', 'e', 'r', 't' };
    char[] input = loginField.getPassword();
    if (Arrays.equals(input, answer)) {
      passLabel.setForeground(Color.green);
      passLabel.setText("Correct!");
      for (int i = 0; i < input.length; i++) input[i] = ' ';
    } else {
      passLabel.setForeground(Color.red);
      passLabel.setText("Wrong!");
    }
    if ((userLabel.getForeground() == Color.green) && (passLabel.getForeground() == Color.
        green)) {
      titleLabel.setText("Logging in.... ");
      login.setEnabled(false);
      userNameField.setEditable(false);
      loginField.setEditable(false);
    }
  }
```



Componente JToogleButton

- Son botones que, a diferencia de los botones normales (**JButton**), cuando los presionas están presionados hasta volver a pulsarlos. Es decir, **se encienden y se apagan con un click**
- Se crean de forma similar a un botón normal

  JToogleButton button = new JToogleButton(String texto);
- Responden a acciones realizadas por el usuario (**Action events**), pero también responden a cambios de estado (**Item events**). Es decir:
  - Cada vez que se selecciona o deselecciona el botón, se produce un cambio de estado. **Este cambio de estado puede ser generado por el usuario o por el propio código**.
- Para capturar eventos del tipo **Item**, la clase debe implementar **ItemListener**, y por tanto el método **itemStateChanged(ItemEvent e)**

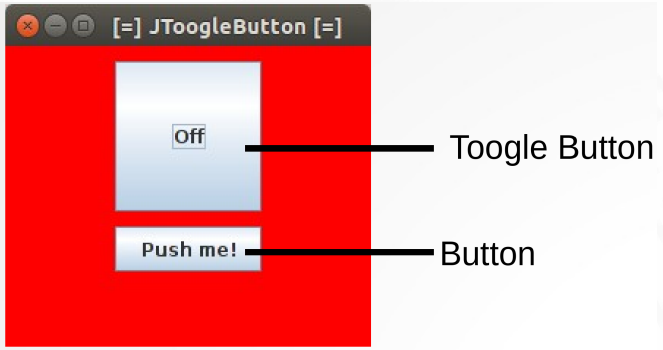
Componente JToogleButton

```
private void initGUI() {
  ...
  toggleButton = new JToggleButton("Off");
  toggleButton.setLocation(75, 10);
  toggleButton.setSize(100, 100);

  toggleButton.addActionListener(this);
  this.add(toggleButton);

  normalButton = new JButton("Push me!");
  normalButton.setLocation(75, 120);
  normalButton.setSize(100, 30);
  normalButton.addActionListener(this);
  this.add(normalButton);
  ...
}

@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == normalButton){
    toggleButton.setSelected(!toggleButton.isSelected());}
  else if (e.getSource()== toggleButton){
    System.out.println("Toggle Button in action listener !!!! ");}
}
```



Componente JToogleButton

```
private void initGUI() {
  ...
  toggleButton = new JToggleButton("Off");
  toggleButton.setLocation(75, 10);
  toggleButton.setSize(100, 100);
  toogleButton.addItemListener(this);
  toggleButton.addActionListener(this);
  this.add(toggleButton);

  normalButton = new JButton("Push me!");
  normalButton.setLocation(75, 120);
  normalButton.setSize(100, 30);
  normalButton.addActionListener(this);
  this.add(normalButton);
  ...
}

@Override
public void itemStateChanged(ItemEvent e) {
  if (e.getStateChange() == ItemEvent.SELECTED) {
    toggleButton.setText("On!");
    this.getContentPane().setBackground(Color.green);
  }
  else {
    toggleButton.setText("Off");
    this.getContentPane().setBackground(Color.red);
  }
}
```

